COST AND PERFORMANCE · DOMAIN 6 - COST & PERFORMANCE OPTIMIZATION

Liquid clustering, OPTIMIZE and VACUUM

Section 6. The layout decision you are allowed to change your mind about.

SECTION 6 of 10 WEIGHT 13% TOPIC Layout

admin@sthithapragna.com

Partitioning is a decision you make once and live with. Liquid clustering is a decision you can **revise**.

What liquid clustering actually is

- A data layout technique that **replaces table partitioning and ZORDER**. The documentation's own word is replaces.
- Files are organised around **clustering keys** rather than a directory structure.
- **Keys can be redefined without rewriting existing data**, so the layout evolves with analytic needs.
- It applies to **streaming tables and materialized views**, not only ordinary tables.

> ON THE EXAM
>
> Databricks recommends liquid clustering for all new tables. That is a blanket recommendation, not a conditional one.

Six scenarios the docs name

- Queries filtering on **high cardinality columns**. Partitioning cannot handle those without shattering the table.
- Tables with **heavy data skew**, where partitions would be wildly uneven.
- **Fast growing tables** that would otherwise need constant maintenance and tuning.
- Tables with **concurrent write** requirements, or **varied and changing access patterns**.
- Tables where a partition key would return **too many or too few partitions**.

> ON THE EXAM
>
> Notice how many of these are exactly the cases where choosing a partition column is hard. That is the point.

You cannot have both

- **Clustering is not compatible with partitioning or ZORDER.** The engine forbids combining them.
- You can enable it on an existing **unpartitioned** table, or at creation time.
- An **already-partitioned** table is not a small change. It needs a migration plan, not an ALTER statement.
- Recall from module 20 that predictive optimization also **ignores Z-ordered files**. The two facts compound.

> ON THE EXAM
>
> A partitioned table cannot simply adopt clustering. That single sentence decides several exam scenarios.

Manual, automatic, and off

FORM	WHAT IT DOES
`CLUSTER BY with columns`	You choose the keys. Up to four, and they must have statistics collected.
`CLUSTER BY AUTO`	Databricks chooses and adapts the keys. Managed tables, and requires predictive optimization.
`CLUSTER BY NONE`	Disables future clustering without rewriting existing data. New writes stay unclustered.

ON THE EXAM

The automatic form needs a Unity Catalog managed table and predictive optimization. On an external table it is simply not available.

How many, and which

- Up to **four** clustering keys. Choose the columns that actually appear in filters and joins.
- Clustering on **too many unrelated keys dilutes** the benefit for every one of them.
- A key with **no statistics collected** cannot drive file skipping, so it does nothing useful.
- If you cannot confidently name the columns, that is an argument for the automatic form rather than a guess.

> ON THE EXAM
>
> The connection to module 21 is direct. Clustering organises files, and statistics are what let the planner skip them.

OPTIMIZE and OPTIMIZE FULL

	WHAT IT DOES
Changing keys	Rewrites nothing. The new keys apply to future clustering only.
OPTIMIZE	Clusters incrementally. New and unclustered data only.
OPTIMIZE FULL	Forces reclustering of all data files. DBR 16.0 and above.

ON THE EXAM

After enabling clustering on a table that already holds data, plain OPTIMIZE will not move it. That is what OPTIMIZE FULL is for.

The one thing partitioning still does better

- **Metadata-only queries over partition columns** can be faster on a partitioned table.
- Selecting the distinct values of a partition column reads directory metadata, not data files.
- A clustered table has to **scan data** to answer the same question.
- So a workload that leans heavily on partition metadata is the honest exception to the recommendation.

ON THE EXAM

Worth knowing so the recommendation does not sound like marketing. Liquid clustering wins nearly everywhere, and not quite everywhere.

The sizing guidance that still applies

- Do not partition tables below roughly **one terabyte**. Smaller tables gain nothing and lose file size.
- If you do partition, aim for at least about **one gigabyte per partition**.
- Under-partitioning produces many tiny files, which is the small file problem OPTIMIZE exists to fix.
- A partition key that yields thousands of tiny partitions is worse than no partitioning at all.

> ON THE EXAM
>
> These thresholds are why so many partitioned tables perform badly. The scheme was chosen for readability rather than file size.

What compaction is actually for

- It rewrites many small files into fewer larger ones, which reduces per-file overhead at read time.
- On a clustered table it also **groups data by the clustering keys**, which is what makes skipping effective.
- Small files accumulate naturally from frequent small writes, particularly streaming ingestion.
- On managed tables, predictive optimization runs this for you when it judges it worthwhile.

> ON THE EXAM
>
> Compaction and clustering are the same command doing two jobs. On an unclustered table it only compacts.

Removing what nothing points at

- It deletes data files **no longer referenced** by any table version inside the retention window.
- Compaction and updates leave old files behind. Without vacuum, storage grows indefinitely.
- **Vacuum is what limits time travel.** Once the files are gone, older versions cannot be read.
- It also **deletes change data feed files**, which is why the feed is not a permanent record.

> ON THE EXAM
>
> Retention defaults to seven days, from module 20. Set it deliberately before enabling predictive optimization if you need longer history.

When you should not run either

- On **Unity Catalog managed tables with predictive optimization**, both operations are run for you.
- A scheduled weekly maintenance job on those tables is **duplicated work** you are paying for twice.
- Check what it decided with **DESCRIBE EXTENDED** before concluding it is not running.
- But remember Z-ordered files are ignored, so a Z-ordered table genuinely does get less.

> ON THE EXAM
>
> The modern answer to 'when should I schedule OPTIMIZE' is usually 'you should not', with two named exceptions.

Runtime versions, and a documentation disagreement

- Generally available for Delta Lake from around **DBR 15.4 LTS**. Public Preview for Apache Iceberg from **16.4 LTS**.
- The documentation is **inconsistent** here. Different pages state 15.4 and 15.2 for the same thing.
- **OPTIMIZE FULL requires DBR 16.0.** That figure is consistent everywhere.
- Managed Iceberg v3 adds deletion vectors, row tracking and automatic clustering, and needs **DBR 18.0**.

> ON THE EXAM
>
> Where documentation disagrees with itself, learn the shape rather than the number. Check your own runtime rather than trusting a course.

What to carry into the exam

- Liquid clustering **replaces partitioning and ZORDER**, and keys can change **without rewriting data**.
- **Not compatible with either.** A partitioned table needs a migration, not an ALTER.
- **OPTIMIZE clusters incrementally. OPTIMIZE FULL reclusters everything.**
- **CLUSTER BY AUTO** needs a managed table and predictive optimization. **CLUSTER BY NONE** stops without rewriting.
- On managed tables, scheduling OPTIMIZE and VACUUM yourself is usually duplicated work.

> ON THE EXAM
>
> Databricks recommends liquid clustering for all new tables. When a question offers a partitioning scheme and a clustering option, the burden of proof is on partitioning.

> A 4 TB fact table is partitioned by transaction date. Analysts increasingly filter on customer identifier, a high cardinality column, and those queries scan almost the whole table. A team member proposes adding liquid clustering on customer identifier with an ALTER TABLE statement.
>
> The table is a Unity Catalog managed table.

What is wrong with the proposal?

A. Nothing. ALTER TABLE with CLUSTER BY adds clustering to any table.

B. Clustering is not compatible with partitioning. The table must be migrated to an unpartitioned form first.

C. Customer identifier is too high cardinality to be a clustering key.

D. Clustering can only be set at table creation, never afterwards.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A. Nothing. ALTER TABLE with CLUSTER BY adds clustering to any table.

Only to an unpartitioned table. The engine forbids combining clustering with partitioning.

B. Clustering is not compatible with partitioning. The table must be migrated to an unpartitioned form first.

Enabling on an existing table is supported only when that table is unpartitioned. This is a migration.

C. Customer identifier is too high cardinality to be a clustering key.

Backwards. High cardinality filter columns are one of the named cases clustering handles well.

D. Clustering can only be set at table creation, never afterwards.

It can be enabled on an existing unpartitioned table.

ON THE EXAM

C is the instructive wrong answer: it applies partitioning intuition to clustering, where high cardinality is a strength rather than a problem.

> A team enables liquid clustering on an existing 900 GB unpartitioned table by setting clustering keys, then runs OPTIMIZE. Query performance improves only slightly, and the query profile shows poor file pruning on older data.
>
> New data written since the change is clustering well.

What should they do?

A. Run OPTIMIZE FULL to force reclustering of all existing data files.

B. Add more clustering keys so more queries benefit.

C. Run VACUUM to remove the old unclustered files.

D. Re-enable Z-ordering alongside clustering for the older data.

◆ PAUSE AND DECIDE

What should they do?

A Run OPTIMIZE FULL to force reclustering of all existing data files.

Plain OPTIMIZE clusters incrementally and leaves existing files alone. OPTIMIZE FULL reclusters everything.

B Add more clustering keys so more queries benefit.

Dilutes the benefit across keys, and does nothing about data that was never clustered.

C Run VACUUM to remove the old unclustered files.

Those files hold current data. Vacuum removes unreferenced files, not referenced ones.

D Re-enable Z-ordering alongside clustering for the older data.

The two cannot coexist, and Z-ordered files are also ignored by predictive optimization.

ON THE EXAM

The symptom is precise: new data clusters well, old data does not. That is incremental clustering behaving exactly as designed.

A platform team wants clustering keys to adapt as query patterns change, without anybody revisiting them. Their tables are Unity Catalog managed with predictive optimization enabled.

A second team wants the same for their tables, which are external tables in cloud storage they manage themselves.

What can each team do?

A Both can use CLUSTER BY AUTO, since it is a table-level setting.

B The first team can use CLUSTER BY AUTO. The second cannot, since it requires a managed table and predictive optimization.

C Neither can; CLUSTER BY AUTO applies only at table creation.

D Both can, but only after running OPTIMIZE FULL first.

◆ PAUSE AND DECIDE

What can each team do?

A Both can use CLUSTER BY AUTO, since it is a table-level setting.

It requires a Unity Catalog managed table. External tables are excluded.

B The first team can use CLUSTER BY AUTO. The second cannot, since it requires a managed table and predictive optimization.

Automatic key selection is driven by predictive optimization, which runs only on managed tables.

C Neither can; CLUSTER BY AUTO applies only at table creation.

It can be set on existing unpartitioned tables too.

D Both can, but only after running OPTIMIZE FULL first.

OPTIMIZE FULL reclusters data. It does not unlock the automatic form.

ON THE EXAM

The second team's realistic path is manual keys now, and converting to managed if they want the automatic form later.

> A reporting workload runs many queries of the form selecting distinct region values, and a dashboard that lists available periods. These run in milliseconds against the current partitioned table.
>
> The team is asked to migrate to liquid clustering, and is concerned about this specific workload.

Is the concern legitimate?

A. No. Clustering is faster than partitioning for every query shape.

B. Yes. Metadata-only queries over partition columns read directory metadata; a clustered table must scan data to answer them.

C. Yes, but only until OPTIMIZE FULL has been run once.

D. No, because clustering keys are stored as metadata and queried the same way.

◆ PAUSE AND DECIDE

Is the concern legitimate?

A. No. Clustering is faster than partitioning for every query shape.

Overstated. The recommendation is broad, not universal.

B. Yes. Metadata-only queries over partition columns read directory metadata; a clustered table must scan data to answer them.

The honest exception. Distinct values of a partition column come from directory names, and clustering has no directories.

C. Yes, but only until OPTIMIZE FULL has been run once.

Reclustering improves skipping. It does not create partition metadata.

D. No, because clustering keys are stored as metadata and queried the same way.

Statistics support file skipping. They are not a distinct-value index.

ON THE EXAM

A rare case where the concern is correct. Naming it makes the broader recommendation credible rather than promotional.

> A team schedules a nightly job running OPTIMIZE and VACUUM across their Unity Catalog managed tables. Half the tables are Z-ordered from an earlier era; the rest are liquid clustered.
>
> They notice the Z-ordered tables perform worse over time despite the nightly job, and the clustered ones perform well.

What best explains the difference?

A The nightly job is misconfigured and skips the Z-ordered tables.

B Predictive optimization ignores Z-ordered files, so those tables get less automatic maintenance. Migrating them to clustering resolves it.

C Z-ordering degrades naturally and must be re-run more frequently.

D VACUUM removes Z-order metadata, undoing the layout each night.

◆ PAUSE AND DECIDE

What best explains the difference?

A The nightly job is misconfigured and skips the Z-ordered tables.

Possible in general, and there is a documented explanation that fits without assuming a bug.

B Predictive optimization ignores Z-ordered files, so those tables get less automatic maintenance. Migrating them to clustering resolves it.

Module 20's finding: OPTIMIZE under predictive optimization does not run ZORDER and ignores Z-ordered files.

C Z-ordering degrades naturally and must be re-run more frequently.

Layouts do drift, and that does not explain a difference against clustered tables under the same job.

D VACUUM removes Z-order metadata, undoing the layout each night.

Vacuum removes unreferenced files. It does not undo a layout.

ON THE EXAM

Two modules converge here. Z-order excludes files from automatic maintenance, and clustering cannot be added alongside it.

> A team wants to stop clustering a table because the workload has changed and the keys no longer match any query. They want new writes to stop being clustered, but they cannot afford a full rewrite of the 12 TB table right now.
>
> They ask whether they must drop and recreate it.

What should they do?

A. Use CLUSTER BY NONE, which disables future clustering without rewriting existing data.

B. Drop and recreate the table without a CLUSTER BY clause.

C. Run OPTIMIZE FULL, which removes clustering when no keys are set.

D. Set the clustering keys to an empty list, which is equivalent to partitioning.

◆ PAUSE AND DECIDE

What should they do?

A Use CLUSTER BY NONE, which disables future clustering without rewriting existing data.

Exactly what it exists for: stop clustering future writes, leave the existing layout in place.

B Drop and recreate the table without a CLUSTER BY clause.

The full rewrite they said they cannot afford, plus the loss of history and permissions.

C Run OPTIMIZE FULL, which removes clustering when no keys are set.

OPTIMIZE FULL reclusters. It is the opposite of what they want.

D Set the clustering keys to an empty list, which is equivalent to partitioning.

Clustering and partitioning are not interchangeable, and the second clause is simply false.

ON THE EXAM

Note what the right answer does not do: it does not undo existing clustering. The old layout stays until something rewrites it.